
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Operation TELESCREEN - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

Project Overview

You are an embedded reverse engineer in the Brotherhood signals lab. The Party has deployed four million TELESCEEN units - combined surveillance cameras and wireless routers - all running the **same node daemon**. The flash from the captured unit did not survive, but the **application binary did**: you hold it as a stripped Linux aarch64 ELF, `firmware/ctfnodestripped`. Your mission is to reverse it, name every function, expose the six deliberate defects (B1-B6), break the weak exfiltration key schedule, and write the hardened replacement.

This is an application-class reverse-engineering project. The target is a real Linux ELF on ARM Cortex-A (aarch64), dynamically linked against glibc. There is **no bare metal**, no Pico, no `.uf2`, and no boot chain to reconstruct. The whole task is the daemon binary and the six logic flaws inside it.

This project covers the application-security arc: artifact identity, dynamic-import recovery, entry-point and dispatcher recovery, function resolution (R1-R4), compiler inlining and phantom functions, unsafe `system()` call sites, a public-identifier key schedule, live defect demonstration in a container, and a real AEAD hardening.

Threat model, stated accurately. Every defect is a **local** flaw: it needs the ability to invoke the daemon or to influence the file/query it is handed. There is no network listener in this binary. "Unauthenticated remote root" is **not** a correct description and will not receive credit.

Learning Objectives

Upon completion of this project, you will demonstrate the ability to:

1. Verify a captured binary by **SHA-256** and by **ELF identity**
2. Recover **library calls** from a stripped binary's `.plt` / `.got` relocations
3. Recover the **entry chain** and a **subcommand dispatcher** from machine code
4. Apply the four **function-resolution rules** (R1-R4) to name every `FUN_`
5. Recognise **compiler inlining** and **phantom functions on alignment padding**
6. Identify unsafe `system()` **sites** and state the correct **threat model**
7. Recover a **reflected CRC-32 key schedule** and explain why a public-identifier key is not encryption
8. Replace a weak KDF with **X25519 + HKDF + AES-256-GCM**
9. Demonstrate each defect locally with the supplied harnesses
10. Analyse the **ethical and legal dimensions** of surveillance-device research

Deliverables Checklist

You must submit **all** of the following. Missing deliverables will result in zero points for the corresponding task.

#	Deliverable	Format	Task
1	Artifact identity: SHA-256 comparison, ELF table, dynamic-import list with the dangerous imports flagged	Inside <code>TELESCREEN-Answers.md</code>	Task 1
2	Command surface: entry chain, subcommand table with string addresses, argument rules	Inside <code>TELESCREEN-Answers.md</code>	Task 2
3	Function map: address -> <code>FUN_</code> -> real name -> rule for all ten application functions, plus the inlined-helper list	Inside <code>TELESCREEN-Answers.md</code>	Task 3
4	Defect catalogue B1-B6 : address, function, the offending call, one-line exploit path, threat-model note	Inside <code>TELESCREEN-Answers.md</code>	Task 4
5	Crypto break: the derivation formula, the sample-UID vector, your reproduction, and a 200-word analysis	Inside <code>TELESCREEN-Answers.md</code>	Task 5
6	Harness output (<code>test_defects.py</code> , <code>test_consistency.py</code>) with a one-line interpretation per check	Inside <code>TELESCREEN-Answers.md</code>	Task 6
7	Hardened replacement: the six fixes plus the AEAD design	Inside <code>TELESCREEN-Answers.md</code>	Task 7
8	Defensive playbook (bonus)	Inside <code>TELESCREEN-Answers.md</code>	Task 8
9	Full submission packaged as <code>lastname-firstname-TELESCREEN.zip</code>	ZIP	-

Required Tools and Equipment

Tool	Purpose	Required For
The instructor-issued <code>firmware/ctfnodestripped</code>	The artifact under analysis	Tasks 1-6

Tool	Purpose	Required For
Docker Desktop / Engine	Runs the aarch64 binary and harnesses	Tasks 5, 6
JDK 21 + Ghidra 12.1.3	Static analysis	Tasks 2-4
<code>readelf</code> , <code>aarch64-linux-gnu-objdump</code> , <code>aarch64-linux-gnu-nm</code> , <code>strings</code>	Relocations, disassembly, symbols, strings	Tasks 1-4
Python 3	The key tool, the harnesses, verification	Tasks 5, 6
A text editor	<code>TELESCREEN-Answers.md</code>	All

No hardware is required. The defects are logic flaws; the harnesses prove them inside the pinned `linux/arm64` container.

Artifact Identity

The instructor-issued artifact hashes are:

```
ctfnode.stripped    af7ab5c2b4837083682db8b54f6892b1a2a33dcc8ce7b202b5a4a5163232da6d
ctfnode.unstripped 6bcee7daa91280eaf02558b1b5f1b5cc8e22a84179605a0725af2b5564257e0a
```

`ctfnode.unstripped` is the instructor answer key and is not part of the student artifact. Verify the stripped hash before you begin; a mismatch means you are analysing the wrong binary.

Grading Rubric - Detailed Breakdown

Task 1: Verify the Artifact (10 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] SHA-256 verified	3	Computed hash matches the instructor-issued value	Hash computed but not compared	Not done
[DOCUMENT] ELF identity correct	3	ELF 64-bit LSB, ARM aarch64, dynamically linked, glibc interpreter, stripped	Class/machine only	Not done
[DOCUMENT] Imports listed	4	All dynamic imports listed; <code>system</code> flagged as the dangerous one for this binary	Imports listed, none flagged	Not done

Task 2: Recover the Command Surface (15 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Entry chain recovered	5	<code>_start -> __libc_start_main -> __wrap_main (0x400874) -> main (0x400800) -> ctf_dispatch (0x400c20)</code>	Two of the hops	Not found

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Subcommand table	6	All six strings (<code>config</code> , <code>http</code> , <code>restore</code> , <code>login</code> , <code>shell</code> , <code>key</code>) with addresses	3-5 correct	0-2 correct
[DOCUMENT] Argument rules	4	Per-subcommand <code>argc</code> requirements read from the <code>cmp</code> / <code>ccmp</code> instructions	Partial	Not found

Task 3: Name Every Function (20 points)

Objective: Name all ten application functions and cite a proving rule for each.

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Address -> name -> rule table	14	All ten functions correctly named, each with a valid rule (R1-R4)	6-9 correct	0-5 correct
[DOCUMENT] Inlined helpers identified	6	All five inlined helpers named and their host functions given	3-4 correct	0-2 correct

The ten functions and their addresses:

address	name
<code>0x400960</code>	<code>ctf_crc32_le</code>
<code>0x4009b0</code>	<code>ctf_weak_key</code>
<code>0x400a8c</code>	<code>ctf_login</code>
<code>0x400ae0</code>	<code>ctf_config_run</code>
<code>0x400b64</code>	<code>ctf_build_cmd</code>
<code>0x400b80</code>	<code>ctf_http_handle</code>
<code>0x400bc0</code>	<code>ctf_restore</code>
<code>0x400c00</code>	<code>ctf_debug_shell</code>
<code>0x400c0c</code>	<code>ctf_banner</code>
<code>0x400c20</code>	<code>ctf_dispatch</code>

Task 4: Prove the Six Defects (30 points)

Objective: Locate and prove B1-B6 with instruction-level evidence.

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] B1	5	<code>ctf_config_run</code> at <code>0x400ae0</code> ; <code>fopen</code> , <code>run=</code> word compare, <code>system</code> all cited	Site only	Not found
[DOCUMENT] B2	5	<code>ctf_build_cmd</code> / <code>ctf_http_handle</code> ; <code>ping -c 1 %s +</code> <code>system</code> cited	One site only	Not found

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] B3	5	ctf_restore at 0x400bc0; tar -xvzf %s -C / + system cited	Site only	Not found
[DOCUMENT] B4	5	ctf_login at 0x400a8c; strcmp(user,"admin") + empty-password test cited	One half	Not found
[DOCUMENT] B5	5	ctf_debug_shell at 0x400c00; system("/bin/sh") + dispatcher path cited	Site only	Not found
[DOCUMENT] B6	5	ctf_weak_key at 0x4009b0; reflected fold + 32-iteration loop cited	Partial	Not found
[DOCUMENT] Threat model	-	Correctly labelled local , not remote; no over-claim	Minor over-claim	"Remote root" claim

The threat-model note is a **correctness gate**: a submission that describes these as unauthenticated remote exploitable cannot exceed 60% on Task 4.

Task 5: Break the Weak Key Schedule (20 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Derivation recovered	8	Polynomial 0xEDB88320, seed 0, initial hash over UID, 32 rounds emitting the low byte; no final XOR	Method named, details wrong	Not found
[PATCH] Sample reproduced	6	SSAT-468547-FEEBD yields da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141	Runs, wrong output	Not submitted
[DOCUMENT] 200-word analysis	6	Explains that a public-identifier key provides no confidentiality	Partial	Not addressed

Task 6: Demonstrate the Defects Locally (15 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[PATCH] test_defects.py run	6	All B1-B5 checks pass; output pasted	Partial run	Not run
[PATCH] test_consistency.py run	5	Binary key, Python tool, and published vector all agree	Partial	Not run
[DOCUMENT] Interpretation	4	One correct sentence per check	Partial	None

Task 7: Write the Hardened Replacement (15 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Six fixes	9	Each defect has a correct, specific fix	3-5 correct	0-2 correct
[DOCUMENT] AEAD design	6	X25519 + HKDF + AES-256-GCM, per-message nonce, tag verification, key source	Partial	Not addressed

Task 8: The Defensive Playbook (bonus, 10 points)

Requirement: A one-page blue-team playbook covering detection and field hardening.

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Output/process detection	4	Names concrete indicators (<code>system</code> call sites, <code>run=</code> config writes, default <code>admin</code> /empty login)	One indicator	Not addressed
[DOCUMENT] Beacon detection	3	Periodicity/entropy analysis with a concrete method	Generic	Not addressed
[DOCUMENT] Field hardening	3	Secure boot, signed updates, secret-key AEAD, read-only rootfs, each justified	Generic	Not addressed

Common Pitfalls

These are the most frequent mistakes. Avoid them.

Pitfall	Consequence	How to Avoid
Calling the defects "remote root"	Factually wrong; capped grade	There is no listener; they are local file/argument flaws
Trusting Ghidra's <code>FUN_</code> labels as if named	You will document guesses	Resolve with R1-R4; cite the rule
Looking for <code>ctf_crc32_byte</code> as a function	You will think the binary is corrupt	It is inlined into <code>ctf_crc32_le</code> and <code>ctf_weak_key</code>
Treating <code>0x400adc</code> as a real function	Your totals and table are wrong	It is a phantom on the alignment pad of <code>0x400ae0</code>
Using a standard CRC-32 (with final XOR) for B6	Key output will not match	The node uses reflected <code>crc32_le</code> with no final inversion
Describing <code>main</code> as the libc entry	Wrong entry chain	<code>__wrap_main (0x400874)</code> is what glibc calls; it jumps to <code>main</code>
Ignoring <code>strcmp</code> in the dispatcher	You will miss the inline login path	Read <code>ctf_dispatch</code> fully; it inlines the small helpers
Reusing a GCM nonce in your fix	Catastrophic forgery	One nonce per message, never reused

The Container Bench

Everything runs in the pinned `linux/arm64` image, so the results are identical on Windows x64, Linux x64, and macOS arm64:

```
./firmware/build_target.sh      # build the target (+ native smoke test)
python3 scripts/test_defects.py # B1-B5 live
python3 scripts/test_consistency.py # B6 vs the Python tool and the vector
```

CRITICAL REMINDER: keep the work **isolated**. Do not point the harnesses at any network or production system; they run only inside the disposable container.

Key Constants Reference

Constant	Value	Meaning
CTF_POLY	0xEDB88320	reflected CRC-32 polynomial
B1_WORD	0x3D6E7572	little-endian <code>run=</code> compare in <code>ctf_config_run</code>
B2_FMT	<code>ping -c 1 %s</code>	format string in <code>ctf_build_cmd</code> / <code>ctf_http_handle</code>
B3_FMT	<code>tar -xvzf %s -C /</code>	format string in <code>ctf_restore</code>
B5_STR	<code>/bin/sh</code>	argument to <code>system</code> in <code>ctf_debug_shell</code>
B4_USER	<code>admin</code>	hard-coded user in <code>ctf_login</code>
B6_ROUNDS	32	key bytes emitted by <code>ctf_weak_key</code>

Deadline & Submission

- This is a **take-home final project**. See the course syllabus for the due date.
- Submission Format:** create a folder holding all deliverables (`TELESCREEN-Answers.md` , the harness output, your key tool, and your hardened design notes).
- ZIP this folder and name it: `lastname-firstname-TELESCREEN.zip` .
 - Example:** if your name is Kevin Thomas, your file must be exactly `thomas-kevin-TELESCREEN.zip` .
- Late submissions: see the syllabus late-work policy.

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86

Grade	Percentage	Points
B-	80-83%	80-83
C	70-79%	70-79
F	0-70%	0-70%

Partial Credit Policy

Scenario	Credit
Correct concept and approach, wrong address or byte value	75% of task points
Identified the function correctly but the evidence is incomplete	60% of task points
Explained the concept correctly but could not locate it in the binary	40% of task points
Fixed correctly but could not explain why the fix works	50% of task points
Documented the analysis process thoroughly even though the result is wrong	30% of task points
Missed one of the ten application functions	85% of Task 3 points
Harnesses run but interpretation is missing	70% of Task 6 points
AEAD chosen but no nonce discipline shown	40% of the AEAD criterion

Academic Integrity

By submitting this final project, you certify that:

1. This is your own work
2. You have not shared answers with other students
3. You understand the ethical implications of embedded security research
4. You understand that the skills demonstrated here may only be used in authorized, lawful contexts
5. You recognize the difference between academic exercises and real-world operations

Reference Material

- ARMv8-A Architecture Reference Manual (A64 instruction set)
- ELF64 and the SysV AArch64 ABI
- Ghidra documentation: <https://ghidra-sre.org/>
- `readelf` / `objdump` documentation; the `.rela.plt` relocation format
- NIST SP 800-38D (GCM); RFC 8439 (ChaCha20-Poly1305); RFC 7748 (X25519)
- OpenIPC / Xiongmai camera-firmware research (prior art)